

TRABAJO PRÁCTICO INTEGRADOR

Asignatura: Programación de Redes con Sockets (Optativa III - LAS)

Cursada: 2026

1. Propósito

El presente trabajo tiene como objetivo integrar los conocimientos y habilidades desarrollados durante la asignatura **Programación de Redes con Sockets**, mediante el diseño e implementación de una aplicación de red basada en el modelo cliente/servidor.

El proyecto deberá involucrar comunicación entre procesos utilizando sockets, protocolos TCP y UDP, manejo de múltiples clientes, validación de datos, tratamiento de errores y la definición de un protocolo de aplicación propio.

La solución deberá ser funcional y demostrable en una red local o mediante la utilización de máquinas virtuales/equipos conectados en red.

2. Situación problemática

Una organización necesita implementar un **Sistema Distribuido de Servicios en Red (SDSR)** que permita a diferentes usuarios conectarse desde computadoras de la organización para consultar información, enviar solicitudes y recibir respuestas de un servidor central.

El sistema deberá permitir que múltiples clientes se conecten simultáneamente al servidor y realicen diferentes operaciones.

La aplicación deberá contemplar dos tipos de comunicación:

- **Comunicación confiable mediante TCP**, destinada a operaciones que requieren garantizar la entrega y el orden de los datos.
- **Comunicación mediante UDP**, destinada a operaciones en las cuales se priorice la rapidez y baja latencia.

Los estudiantes deberán definir el problema concreto que resolverá la aplicación. Algunos ejemplos posibles son:

- Sistema de mensajería cliente/servidor.
- Sistema de monitoreo de equipos de una red.
- Sistema de consulta y actualización de información.
- Sistema de transferencia de archivos.

- Sistema de gestión de recursos compartidos.
- Sistema de monitoreo de sensores.
- Sistema de consultas sobre una base de datos.
- Sistema de notificaciones y eventos.
- Juego distribuido simple.
- Sistema de administración remota de información.

La elección del dominio deberá ser justificada en el informe.

3. Objetivo general

Diseñar e implementar una aplicación distribuida cliente/servidor que utilice programación mediante sockets y que integre los conceptos, técnicas y herramientas abordados durante la asignatura.

4. Objetivos específicos

El proyecto deberá permitir al estudiante:

1. Diseñar una arquitectura cliente/servidor.
2. Implementar comunicación utilizando sockets TCP.
3. Implementar comunicación utilizando sockets UDP.
4. Definir un protocolo de aplicación propio.
5. Implementar un servidor capaz de atender múltiples clientes.
6. Incorporar mecanismos de concurrencia.
7. Implementar validación de los datos recibidos.
8. Incorporar mecanismos adecuados de tratamiento de errores.
9. Analizar el comportamiento de la aplicación sobre una red.
10. Identificar las direcciones IP y puertos utilizados.
11. Analizar las comunicaciones generadas por la aplicación mediante una herramienta de captura de tráfico.
12. Considerar aspectos básicos de seguridad en la comunicación.
13. Documentar las decisiones de diseño e implementación.
14. Evaluar las ventajas y limitaciones de las soluciones desarrolladas.

5. Requerimientos funcionales mínimos

La aplicación deberá cumplir, como mínimo, con los siguientes requerimientos.

RF01 – Servidor

Deberá existir un programa servidor encargado de recibir y procesar las solicitudes de los clientes.

El servidor deberá:

- Inicializar los sockets correspondientes.
- Escuchar solicitudes de conexión.
- Aceptar clientes TCP.
- Recibir y procesar solicitudes.
- Generar respuestas.
- Registrar eventos importantes de la comunicación.
- Manejar adecuadamente la desconexión de los clientes.

RF02 – Cliente

Deberá existir un programa cliente que permita al usuario:

- Conectarse al servidor.
- Identificar el servidor mediante dirección IP y puerto.
- Enviar solicitudes.
- Recibir respuestas.
- Mostrar los resultados al usuario.
- Detectar y comunicar errores de comunicación.

RF03 – Comunicación TCP

Al menos una funcionalidad principal del sistema deberá utilizar **TCP**.

La comunicación deberá contemplar:

- Establecimiento de conexión.
- Envío de solicitudes.
- Recepción de respuestas.
- Cierre correcto de la conexión.
- Manejo de errores.
- Detección de desconexiones.

RF04 – Comunicación UDP

Al menos una funcionalidad deberá implementarse mediante **UDP**.

El grupo deberá justificar por qué dicha funcionalidad resulta apropiada para UDP y no necesariamente para TCP.

Deberá analizarse, como mínimo:

- Ausencia de establecimiento de conexión.
- Envío y recepción de datagramas.
- Dirección IP y puerto de destino.
- Posibilidad de pérdida de datagramas.
- Limitaciones de confiabilidad.

RF05 – Múltiples clientes

El servidor deberá ser capaz de atender **al menos tres clientes simultáneamente**.

La solución deberá utilizar algún mecanismo de concurrencia, por ejemplo:

- Hilos.
- Procesos.
- Multiplexación de E/S.
- Otro mecanismo técnicamente justificado.

El estudiante deberá explicar el mecanismo seleccionado y justificar su utilización.

RF06 – Protocolo de aplicación

El grupo deberá diseñar un **protocolo de aplicación propio** para la comunicación entre cliente y servidor.

El protocolo deberá definir, como mínimo:

- Formato de los mensajes.
- Tipos de solicitudes.
- Formato de las respuestas.
- Códigos de operación.
- Códigos de error.
- Reglas de validación.
- Secuencia de intercambio de mensajes.

Por ejemplo:

LOGIN|usuario|clave

Respuesta:

OK|LOGIN

o

ERROR|AUTH|Usuario inexistente

El protocolo podrá utilizar texto plano, JSON, estructuras binarias u otro formato debidamente justificado.

6. Manejo de errores

La aplicación deberá contemplar situaciones anormales, entre ellas:

- Servidor no disponible.
- Puerto ocupado.
- Cliente desconectado inesperadamente.
- Datos inválidos.
- Solicitud desconocida.
- Timeout.
- Error durante el envío o recepción.
- Intento de acceso no autorizado.
- Mensajes incompletos o mal formados.

Los errores deberán ser tratados de manera controlada, evitando la terminación inesperada de la aplicación.

7. Validación de datos

El servidor **no deberá confiar directamente en los datos recibidos desde los clientes.**

Deberá validar, según corresponda:

- Tipo de operación.
- Cantidad y formato de parámetros.
- Longitud de mensajes.
- Valores permitidos.
- Identificación del usuario.
- Integridad lógica de las solicitudes.

8. Seguridad básica (opcional)

La aplicación deberá incorporar al menos una medida básica de seguridad.

Por ejemplo:

- Autenticación de usuarios.
- Control de acceso.

- Validación estricta de solicitudes.
- Protección de información sensible.
- Cifrado simple.
- Uso conceptual de TLS/SSL.

No se requiere implementar una infraestructura criptográfica completa, pero sí identificar los principales riesgos de seguridad de la solución y proponer mecanismos para mitigarlos.

9. Análisis de tráfico de red

Una parte fundamental del trabajo será demostrar qué ocurre realmente en la red durante la ejecución de la aplicación.

Para ello deberá utilizarse una herramienta de captura y análisis de tráfico, por ejemplo **Wireshark**.

El grupo deberá capturar al menos:

1. Una comunicación TCP.
2. Una comunicación UDP.
3. La comunicación entre un cliente y el servidor.
4. La comunicación cuando existen múltiples clientes.

Deberán identificarse y documentarse:

- Dirección IP origen.
- Dirección IP destino.
- Puerto origen.
- Puerto destino.
- Protocolo utilizado.
- Tamaño de los mensajes.
- Secuencia de comunicación.
- Establecimiento de la conexión TCP.
- Intercambio de datos.
- Finalización de la conexión.

Cuando corresponda, deberán analizarse los segmentos TCP y datagramas UDP observados.

10. Comparación TCP vs. UDP

El grupo deberá realizar una experiencia que permita comparar el comportamiento de TCP y UDP en las funcionalidades implementadas.

Se deberá analizar:

Característica	TCP	UDP
Orientación a conexión		
Confiabilidad		
Orden de los datos		
Control de flujo		
Latencia		
Sobrecarga		
Tratamiento de pérdidas		
Aplicación desarrollada		

Las conclusiones deberán basarse tanto en los conceptos teóricos como en las pruebas realizadas.

11. Requerimientos técnicos

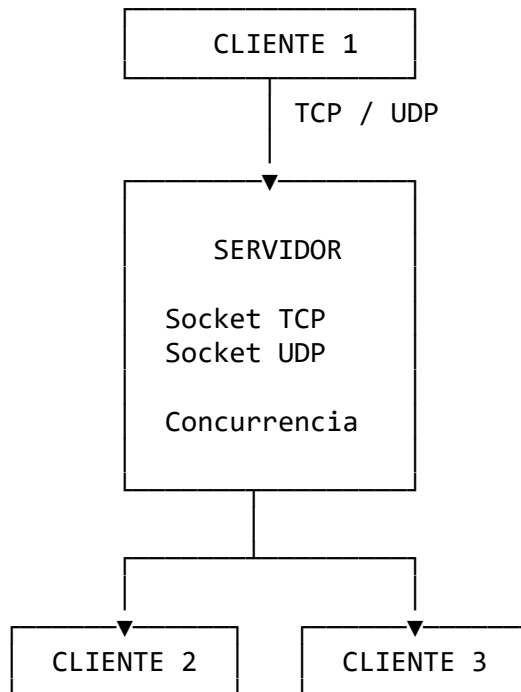
La aplicación deberá cumplir con las siguientes condiciones:

- Utilizar programación mediante sockets.
- Implementar arquitectura cliente/servidor.
- Utilizar TCP y UDP.
- Permitir múltiples clientes concurrentes.
- Utilizar direccionamiento IP y puertos configurables.
- Implementar un protocolo de aplicación.
- Implementar validación de datos.
- Implementar manejo de errores.
- Registrar eventos relevantes.
- Poder ejecutarse en al menos dos equipos o máquinas virtuales conectadas mediante red.

El lenguaje de programación será el utilizado durante el cursado de la asignatura o aquel autorizado por el equipo docente.

12. Arquitectura

El informe deberá incluir un diagrama de arquitectura que represente, como mínimo:



El diagrama deberá adaptarse a la solución desarrollada.

13. Pruebas

Se deberán realizar pruebas funcionales y de comunicación.

Como mínimo:

Prueba 1 – Cliente único

Verificar la comunicación normal entre un cliente y el servidor.

Prueba 2 – Múltiples clientes

Conectar simultáneamente al menos tres clientes.

Prueba 3 – Desconexión

Desconectar un cliente durante una operación y verificar el comportamiento del servidor.

Prueba 4 – Datos inválidos

Enviar solicitudes incorrectas y comprobar que el servidor las rechace adecuadamente.

Prueba 5 – Servidor no disponible

Intentar conectar un cliente cuando el servidor no está funcionando.

Prueba 6 – TCP

Analizar mediante Wireshark una comunicación TCP.

Prueba 7 – UDP

Analizar mediante Wireshark una comunicación UDP.

Prueba 8 – Concurrencia

Demostrar que un cliente puede realizar una operación mientras otros clientes son atendidos.

14. Informe final

El grupo deberá entregar un informe técnico que contenga:

1. Portada.
2. Integrantes.
3. Introducción.
4. Descripción del problema.
5. Objetivos.
6. Requerimientos funcionales y no funcionales.
7. Arquitectura de la solución.
8. Diseño del protocolo de aplicación.
9. Descripción de la implementación.
10. Implementación TCP.
11. Implementación UDP.
12. Mecanismo de concurrencia.
13. Manejo de errores.
14. Validación de datos.
15. Seguridad.
16. Pruebas realizadas.
17. Análisis de tráfico mediante Wireshark.
18. Comparación TCP/UDP.

- 19. Resultados obtenidos.
- 20. Problemas encontrados y soluciones adoptadas.
- 21. Conclusiones.
- 22. Bibliografía.
- 23. Anexos.

15. Entregables

Cada grupo deberá entregar:

A. Código fuente

Código completo del cliente y servidor.

B. Documentación

Informe técnico en formato PDF.

C. Protocolo

Documento donde se especifique formalmente el protocolo de aplicación diseñado.

D. Evidencias

Capturas de pantalla y/o archivos de captura de tráfico que permitan comprobar el funcionamiento.

E. Video demostrativo

Video breve, de aproximadamente **5 a 10 minutos**, mostrando:

- Inicio del servidor.
- Conexión de clientes.
- Comunicación TCP.
- Comunicación UDP.
- Atención simultánea de varios clientes.
- Alguna situación de error.
- Captura/análisis del tráfico de red.

16. Presentación y defensa

Cada grupo deberá realizar una demostración de la aplicación.

Durante la defensa, los docentes podrán solicitar modificaciones o pruebas adicionales, tales como:

- Conectar un nuevo cliente.
- Cambiar la dirección IP.
- Cambiar el puerto.
- Provocar una desconexión.
- Enviar información inválida.
- Explicar un segmento TCP.
- Explicar un datagrama UDP.
- Modificar una funcionalidad del protocolo.
- Explicar cómo se resuelve la concurrencia.

Cada integrante deberá demostrar conocimiento suficiente del código y de las decisiones de diseño adoptadas.

17. Condición de aprobación

Para aprobar el trabajo integrador, la aplicación deberá ser **funcional**, cumplir los requerimientos mínimos establecidos y ser defendida satisfactoriamente por los integrantes del grupo.

La presentación de código que no pueda ser explicado por sus autores podrá ser considerada insuficiente.

La evaluación tendrá en cuenta especialmente la capacidad del estudiante para **comprender, justificar, implementar, probar y analizar** la solución desarrollada, y no solamente el resultado final del programa.

18. Trabajo grupal

El trabajo podrá realizarse en grupos de **hasta tres estudiantes**.

Todos los integrantes deberán participar en:

- Diseño.
- Implementación.
- Pruebas.
- Documentación.
- Defensa.

Cada integrante deberá poder explicar el funcionamiento general del sistema y particularmente las partes que haya desarrollado.

19. Desafío adicional – opcional

Los grupos que deseen ampliar la propuesta podrán incorporar funcionalidades adicionales, tales como:

- Multiplexación mediante `select()` o `poll()`.
- Comunicación asincrónica.
- Transferencia de archivos.
- Persistencia mediante una base de datos.
- Sistema de autenticación más robusto.
- Cifrado de comunicaciones.
- Implementación conceptual de TLS/SSL.
- Registro de actividad.
- Sistema de recuperación ante fallos.
- Balanceo entre múltiples servidores.
- Interfaz gráfica.
- API o integración con otro servicio.
- Medición de latencia y rendimiento.
- Implementación de un protocolo binario.

Estas extensiones no reemplazan los requerimientos mínimos del trabajo.